

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа.

Тема: «Внешние сортировки»

По дисциплине «Основы алгоритмизации и программирования»

Выполнила: студентка группы РИС-25-26

Абрамова Валерия Андреевна

Принял: доц. Полякова О.А.

Пермь, 2026

Сортировка слияния

Постановка задачи

Отсортировать массив целых чисел методом естественного слияния, который находит уже существующие упорядоченные подпоследовательности (серии) и последовательно их сливает, пока массив не будет полностью отсортирован.

Анализ задачи

В отличие от обычной сортировки слиянием, где массив искусственно делится на равные части, естественное слияние использует уже имеющиеся в массиве упорядоченные фрагменты. Это делает алгоритм адаптивным — он эффективнее работает с частично отсортированными данными. Исходный массив из 8 чисел сначала разбивается на 5 естественных серий разной длины. Затем последовательно сливаются соседние серии, причём нечётная последняя серия остаётся без пары. После каждого прохода количество серий уменьшается. За три прохода массив полностью сортируется.

Код

```
#include <iostream>
#include <locale>
using namespace std;

void print(int a[], int n, string s = "") {
    cout << s;
    for (int i = 0; i < n; i++) cout << a[i] << " ";
    cout << endl;
}

int runs(int a[], int n, int b[]) {
    int c = 0;
    for (int i = 1; i <= n; i++)
        if (i == n || a[i] < a[i - 1]) b[c++] = i;
    return c;
}

void merge(int a[], int t[], int l, int m, int r) {
    int i = l, j = m, k = l;
    while (i < m && j < r) t[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    while (i < m) t[k++] = a[i++];
    while (j < r) t[k++] = a[j++];
    for (int idx = l; idx < r; idx++) a[idx] = t[idx];
}

void natSort(int a[], int n) {
    int* t = new int[n];
    int b[100];
    int step = 1;

    print(a, n, "Исходный: ");

    while (true) {
```

```

int cnt = runs(a, n, b);
cout << "\nШаг " << step++ << "\nСерий: " << cnt << "\nГраницы: ";
for (int i = 0; i < cnt; i++) cout << b[i] << " ";
cout << endl;

if (cnt == 1) { cout << "Отсортировано!\n"; break; }

for (int i = 0; i < cnt - 1; i += 2) {
    int l = (i == 0) ? 0 : b[i - 1];
    int m = b[i];
    int r = b[i + 1];
    merge(a, t, l, m, r);
    print(a, n, " После слияния: ");
}
if (cnt % 2 == 1) cout << " Последняя серия без изменений\n";
}

print(a, n, "Отсортированный: ");
delete[] t;
}

int main() {
    setlocale(LC_ALL, "ru");

    int arr[] = { 41, 9, 15, 24, 2, 1, 12, 7 };
    natSort(arr, 8);
    return 0;
}

```

Вывод программы

```

Исходный: 41 9 15 24 2 1 12 7

Шаг 1
Серий: 5
Границы: 1 4 5 7 8
  После слияния: 9 15 24 41 2 1 12 7
  После слияния: 9 15 24 41 1 2 12 7
  Последняя серия без изменений

Шаг 2
Серий: 3
Границы: 4 7 8
  После слияния: 1 2 9 12 15 24 41 7
  Последняя серия без изменений

Шаг 3
Серий: 2
Границы: 7 8
  После слияния: 1 2 7 9 12 15 24 41

Шаг 4
Серий: 1
Границы: 8
Отсортировано!
Отсортированный: 1 2 7 9 12 15 24 41

```

Сбалансированное двухфазное слияние

Постановка задачи

Реализовать сбалансированную сортировку слиянием (классическое рекурсивное слияние) для целочисленного массива.

Анализ задачи

1. Исходное множество целых чисел делится на две части. При нечётном количестве элементов лишний элемент добавляется к левому подмножеству.
2. Деление повторяется рекурсивно до тех пор, пока в каждом подмножестве не останется не более двух элементов.
3. Каждое подмножество сортируется (например, по возрастанию).
4. Отсортированные подмножества попарно сливаются в единое упорядоченное множество.
5. Процесс сортировки и слияния повторяется до получения полностью отсортированного множества.
6. Результат — отсортированное множество.

Код

```
#include <locale>
#include <iostream>
using namespace std;

void print(int a[], int n, string s = "") {
    cout << s;
    for (int i = 0; i < n; i++) cout << a[i] << " ";
    cout << endl;
}

void merge(int a[], int l, int m, int r) {
    int n1 = m - l + 1;
    int n2 = r - m;

    int* L = new int[n1];
    int* R = new int[n2];

    for (int i = 0; i < n1; i++) L[i] = a[l + i];
    for (int j = 0; j < n2; j++) R[j] = a[m + 1 + j];

    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2) a[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i < n1) a[k++] = L[i++];
    while (j < n2) a[k++] = R[j++];

    delete[] L;
    delete[] R;
}

void sort(int a[], int l, int r, int& step, int n) {
    if (l < r) {
        int m = l + (r - l) / 2;
```

```

        cout << "\nШаг " << step++ << "] РАЗДЕЛЕНИЕ [" << l << "-" << r << "]" << endl;
        cout << "    Левая: [" << l << "-" << m << "]" << endl;
        cout << "    Правая: [" << m + 1 << "-" << r << "]" << endl;

        sort(a, l, m, step, n);
        sort(a, m + 1, r, step, n);

        cout << "\nШаг " << step++ << "] СЛИЯНИЕ [" << l << "-" << m << "]" + [" << m +
1 << "-" << r << "]" << endl;

        cout << "    Левый: ";
        for (int i = l; i <= m; i++) cout << a[i] << " ";
        cout << endl << "    Правый: ";
        for (int i = m + 1; i <= r; i++) cout << a[i] << " ";
        cout << endl;

        merge(a, l, m, r);

        cout << "    Результат: ";
        print(a, n);
    }
}

int main() {
    setlocale(LC_ALL, "ru");

    int a[] = { 6, 2, 3, 7, 1, 9, 5, 8, 4 };
    int n = 9;

    cout << "Исходный: ";
    print(a, n);

    int step = 1;
    sort(a, 0, n - 1, step, n);

    cout << "\nИтоговый: ";
    print(a, n);

    return 0;
}

```

Вывод программы

Исходный: 6 2 3 7 1 9 5 8 4	[Шаг 9] СЛИЯНИЕ [0-2] + [3-4] Левый: 2 3 6 Правый: 1 7 Результат: 1 2 3 6 7 9 5 8 4
[Шаг 1] РАЗДЕЛЕНИЕ [0-8] Левая: [0-4] Правая: [5-8]	[Шаг 10] РАЗДЕЛЕНИЕ [5-8] Левая: [5-6] Правая: [7-8]
[Шаг 2] РАЗДЕЛЕНИЕ [0-4] Левая: [0-2] Правая: [3-4]	[Шаг 11] РАЗДЕЛЕНИЕ [5-6] Левая: [5-5] Правая: [6-6]
[Шаг 3] РАЗДЕЛЕНИЕ [0-2] Левая: [0-1] Правая: [2-2]	[Шаг 12] СЛИЯНИЕ [5-5] + [6-6] Левый: 9 Правый: 5 Результат: 1 2 3 6 7 5 9 8 4
[Шаг 4] РАЗДЕЛЕНИЕ [0-1] Левая: [0-0] Правая: [1-1]	[Шаг 13] РАЗДЕЛЕНИЕ [7-8] Левая: [7-7] Правая: [8-8]
[Шаг 5] СЛИЯНИЕ [0-0] + [1-1] Левый: 6 Правый: 2 Результат: 2 6 3 7 1 9 5 8 4	[Шаг 14] СЛИЯНИЕ [7-7] + [8-8] Левый: 8 Правый: 4 Результат: 1 2 3 6 7 5 9 4 8
[Шаг 6] СЛИЯНИЕ [0-1] + [2-2] Левый: 2 6 Правый: 3 Результат: 2 3 6 7 1 9 5 8 4	[Шаг 15] СЛИЯНИЕ [5-6] + [7-8] Левый: 5 9 Правый: 4 8 Результат: 1 2 3 6 7 4 5 8 9
[Шаг 7] РАЗДЕЛЕНИЕ [3-4] Левая: [3-3] Правая: [4-4]	[Шаг 16] СЛИЯНИЕ [0-4] + [5-8] Левый: 1 2 3 6 7 Правый: 4 5 8 9 Результат: 1 2 3 4 5 6 7 8 9
[Шаг 8] СЛИЯНИЕ [3-3] + [4-4] Левый: 7 Правый: 1 Результат: 2 3 6 1 7 9 5 8 4	Итоговый: 1 2 3 4 5 6 7 8 9
[Шаг 9] СЛИЯНИЕ [0-2] + [3-4] Левый: 2 3 6 Правый: 1 7 Результат: 1 2 3 6 7 9 5 8 4	

Многофазная сортировка

Постановка задачи

Реализовать многофазную сортировку для целочисленного массива.

Анализ задачи

1. Исходный массив разбивается на две "ленты" (вспомогательных массива) Чётные индексы (0, 2, 4...) попадают в Ленту 1 Нечётные индексы (1, 3, 5...) попадают в Ленту 2
2. Каждая лента сортируется независимо (в коде используется сортировка пузырьком)
3. Лента 1 становится отсортированной по возрастанию Лента 2 становится отсортированной по возрастанию. Две отсортированные ленты сливаются в третью ленту. Слияние происходит стандартным алгоритмом: сравниваются текущие элементы из обеих лент, меньший идёт в результат
4. Отсортированная третья лента копируется обратно в исходный массив

Код

```
#include <iostream>
#include <locale>
using namespace std;

const int MX = 100;

void print(int a[], int n, string s = "") {
    cout << s;
    for (int i = 0; i < n; i++) cout << a[i] << " ";
    cout << endl;
}

int merge(int a[], int na, int b[], int nb, int res[]) {
    int i = 0, j = 0, k = 0;
    while (i < na && j < nb) res[k++] = (a[i] <= b[j]) ? a[i++] : b[j++];
    while (i < na) res[k++] = a[i++];
    while (j < nb) res[k++] = b[j++];
    return k;
}

void sort(int a[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = i + 1; j < n; j++)
            if (a[i] > a[j]) swap(a[i], a[j]);
}

void multiSort(int a[], int n) {
    int t1[MX], t2[MX], t3[MX];
    int s1 = 0, s2 = 0;

    print(a, n, "Исходный: ");

    for (int i = 0; i < n; i++) {
        if (i % 2 == 0) t1[s1++] = a[i];
        else t2[s2++] = a[i];
    }

    cout << "\nРаспределение\nЛента 1: ";
    print(t1, s1);
    cout << "Лента 2: ";
    print(t2, s2);

    sort(t1, s1);
    sort(t2, s2);

    cout << "\nПосле сортировки:\nЛента 1: ";
    print(t1, s1);
    cout << "Лента 2: ";
    print(t2, s2);
    int s3 = merge(t1, s1, t2, s2, t3);
    cout << "\nСлияние\nЛента 3: ";
    print(t3, s3);

    for (int i = 0; i < s3; i++) a[i] = t3[i];
    print(a, n, "\nОтсортирован: ");
}

int main() {
    setlocale(LC_ALL, "ru");

    int a[] = { 15, 3, 8, 1, 12, 7, 4, 10, 5, 9, 2, 6, 11, 13, 14 };
    int n = sizeof(a) / sizeof(a[0]);

    multiSort(a, n);
    return 0;
}
```

Вывод программы

 Консоль отладки Microsoft Visual Studio

Исходный: 15 3 8 1 12 7 4 10 5 9 2 6 11 13 14

Распределение

Лента 1: 15 8 12 4 5 2 11 14

Лента 2: 3 1 7 10 9 6 13

После сортировки:

Лента 1: 2 4 5 8 11 12 14 15

Лента 2: 1 3 6 7 9 10 13

Слияние

Лента 3: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

Отсортирован: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

C:\Users\Валерия\source\repos\ConsoleApplication36\x64\Debug\ConsoleApp1
Нажмите любую клавишу, чтобы закрыть это окно: